Full Stack Development with MERN

Project Documentation format

1. Introduction

- Project Title: Qcab – Online Cab Booking Application
- Team Members:

- 2. Project Overview

- Purpose: Qcab is a MERN-based online cab booking web application that lets customers register, log in, book a cab by entering pickup and drop locations, view the fare, and track their booking history — while giving administrators a dedicated panel to manage users, cabs and bookings.
- Features: User registration and login; cab booking with pickup/drop entry and car selection; automatic fare display and driver assignment; My Bookings history; Admin login; Admin dashboard with system totals and recent cabs; Admin management of users; Admin management of cabs (add, edit, delete).

3. Architecture

- Frontend: Built with React.js. Pages/components include the public landing page, Login/Register, Home, Book Cab, My Bookings, and the Admin panel (Dashboard, Users, Cabs, Add Cab, Bookings). React Router handles navigation and Axios calls the backend REST API; the UI is styled with Bootstrap.
- Backend: Built with Node.js and Express.js, exposing REST APIs for authentication, cab booking, and admin management (users, cabs, bookings). Express middleware handles JWT verification and role-based access for user vs admin routes.
- Database: MongoDB (via Mongoose) with three main collections — Users (name, email, password hash, role), Cabs (car name, type, driver name, fare per km, plate number, image), and Bookings (user, cab, pickup, drop, fare, booking date).

4. Setup Instructions

- Prerequisites: Node.js and npm, MongoDB (local instance or MongoDB Atlas), and Git.
- Installation: Clone the repository, then run npm install inside both the client and server folders.
- Then create a .env file in the server folder with MONGO_URI, JWT_SECRET and PORT, and start MongoDB before running the servers.

5. Folder Structure

- Client: The client/ folder contains the React app — src/components (reusable UI such as Navbar, Footer, Car cards), src/pages (Home, Login, Register, BookCab, MyBookings, and the Admin pages), App.js for routing, and index.js as the entry point.
- Server: The server/ folder contains models/ (Mongoose schemas for User, Cab, Booking), routes/ and controllers/ (auth, cabs, bookings, users), middleware/ (JWT authentication), and server.js as the Express entry point.

6. Running the Application

- Run the following commands in two separate terminals to start Qcab locally:

o Frontend: npm start in the client directory (runs on http://localhost:3000).

o Backend: npm start in the server directory (runs on http://localhost:5000).

7. API Documentation

- Qcab exposes the following REST endpoints:

Example: POST /api/auth/login expects { email, password } and returns a signed JWT token with the user's role.

- POST /api/auth/register — register a new user
- POST /api/auth/login — user login, returns a JWT
- POST /api/admin/login — admin login
- GET /api/cabs — list all available cabs
- POST /api/cabs — add a new cab (admin)
- PUT /api/cabs/:id — edit a cab (admin)
- DELETE /api/cabs/:id — delete a cab (admin)
- POST /api/bookings — create a new booking
- GET /api/bookings/:userId — get a user's booking history
- GET /api/admin/users — list all users (admin)
- DELETE /api/admin/users/:id — delete a user (admin)
- GET /api/admin/dashboard — system totals for users, cabs and bookings

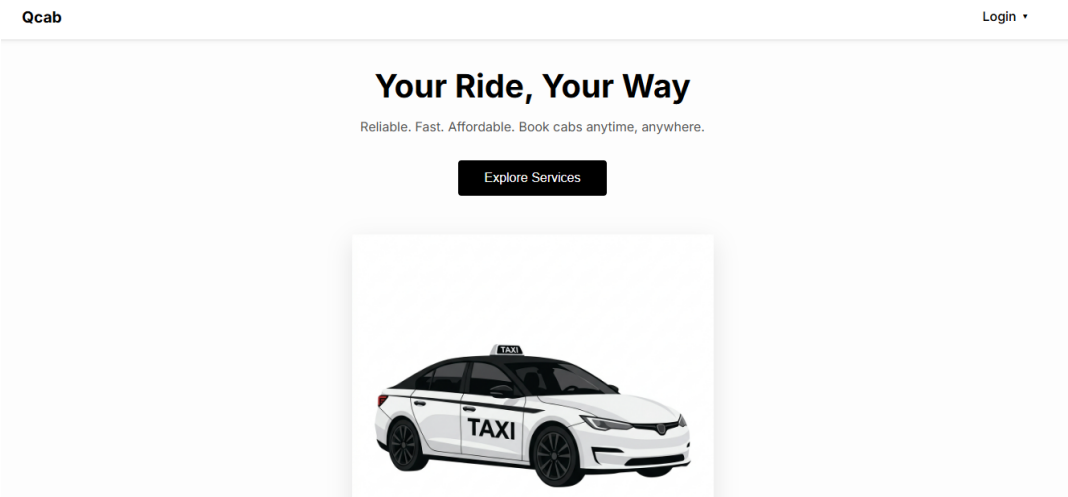
8. Authentication

- User and admin passwords are hashed with bcrypt before being stored. On login, the server verifies the password and issues a signed JWT.
- The JWT is returned to the client, stored in localStorage, and sent as a Bearer token in the Authorization header on protected requests; middleware on the server verifies the token and checks the user's role (user vs admin) before allowing access.

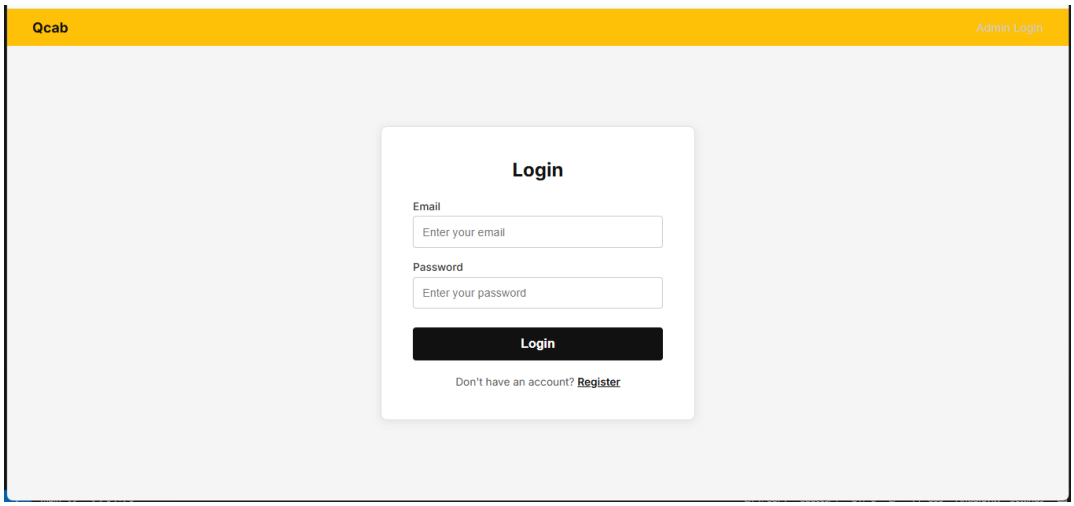
9. User Interface

- Key screens of the Qcab web application (user side):

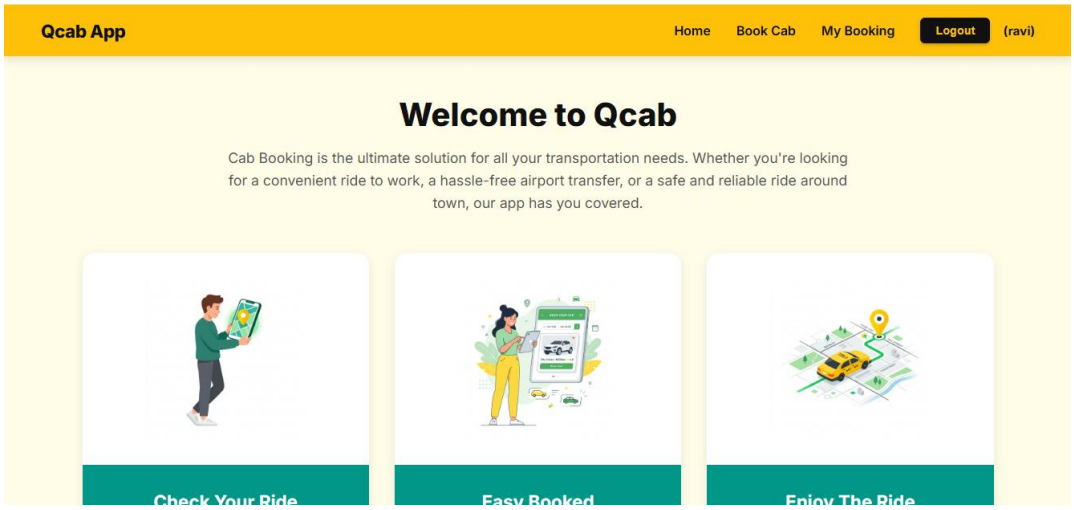
1. Public Landing Page — “Your Ride, Your Way”



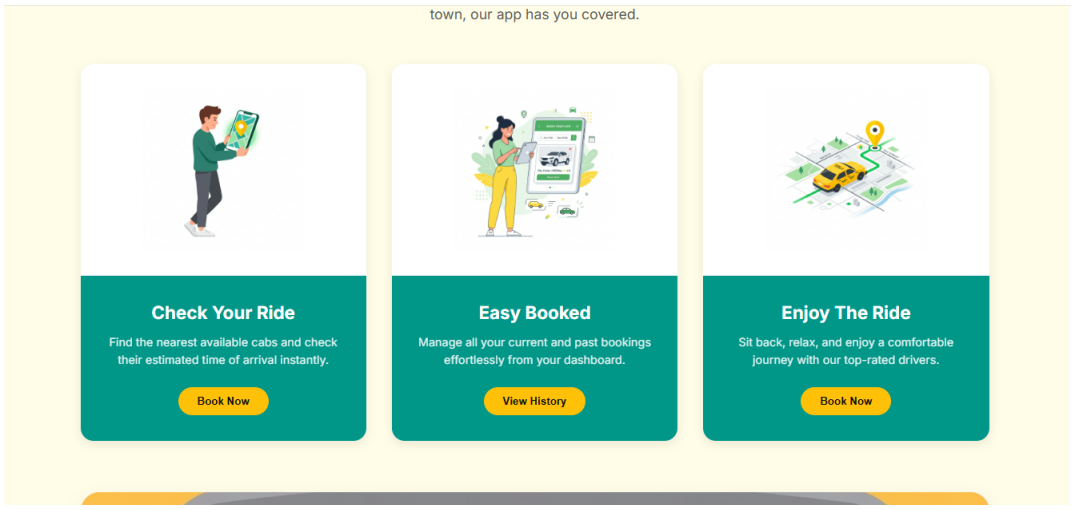
2. Login — User Login Screen



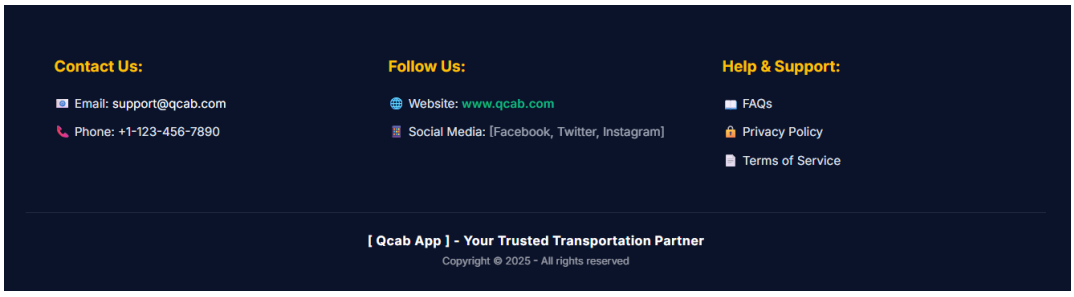
3. Home (after login) — “Welcome to Qcab”



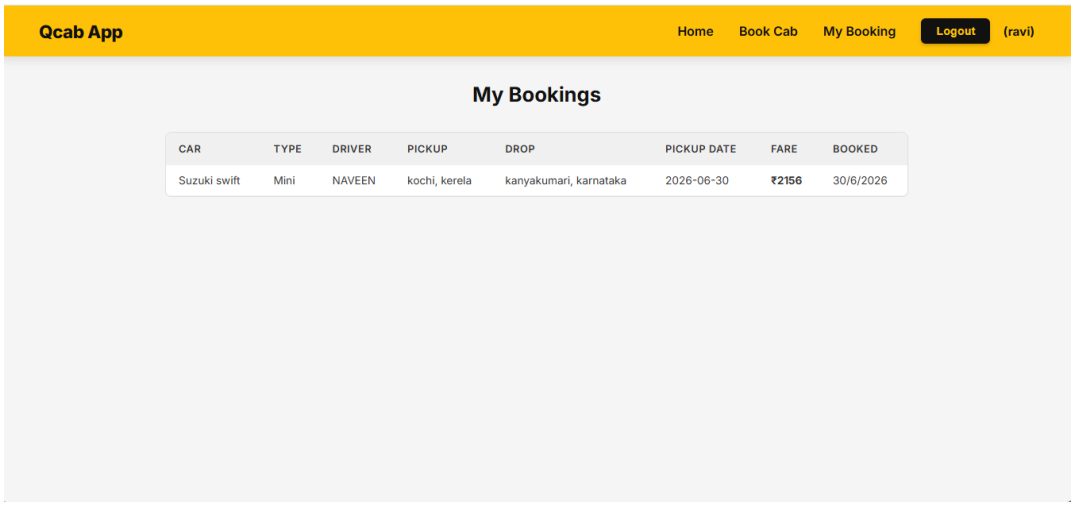
4. Home — Feature Highlights



5. Footer — Contact & Support



6. My Bookings — Ride History



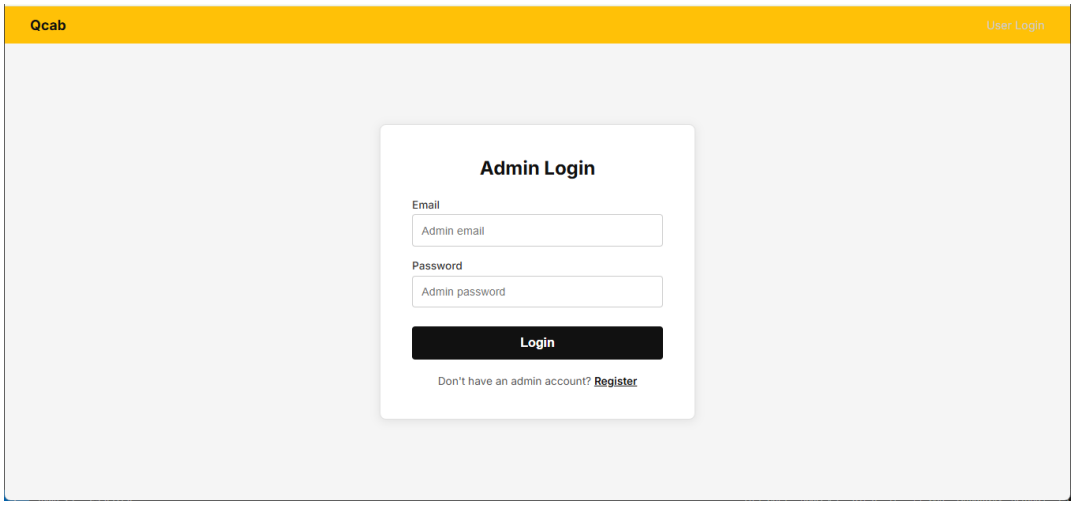
10. Testing

- Qcab was tested manually through User Acceptance Testing (UAT) covering registration, login, cab booking, fare display, booking history, and the admin panel (dashboard, users, cabs). Test cases, expected results and pass/fail status are documented in the separate UAT report.

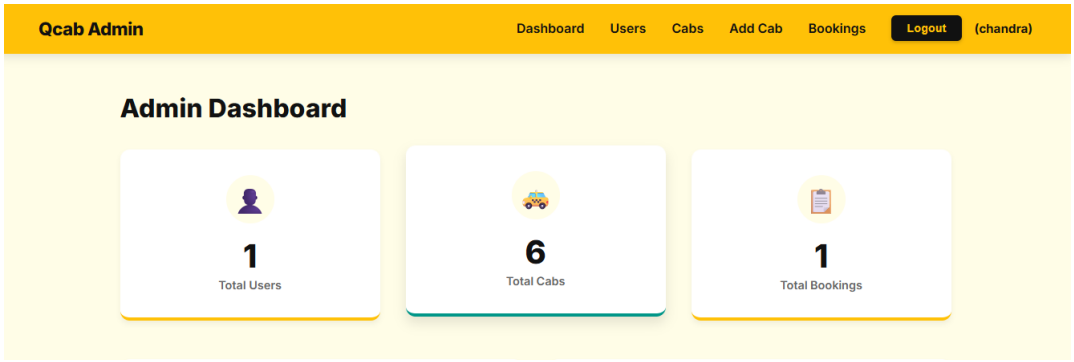
11. Screenshots or Demo

- Key screens of the Qcab admin panel:

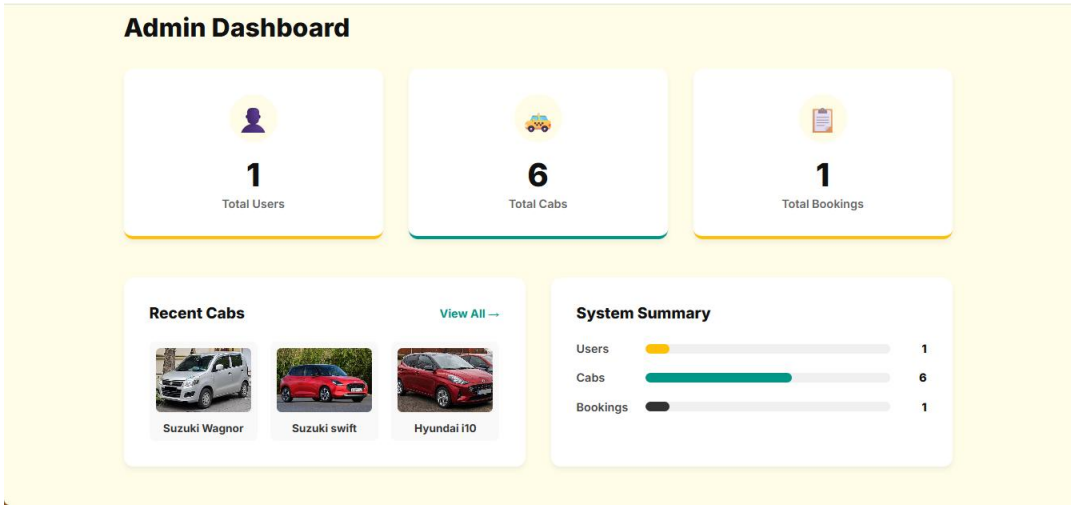
1. Admin Login — Admin Login Screen



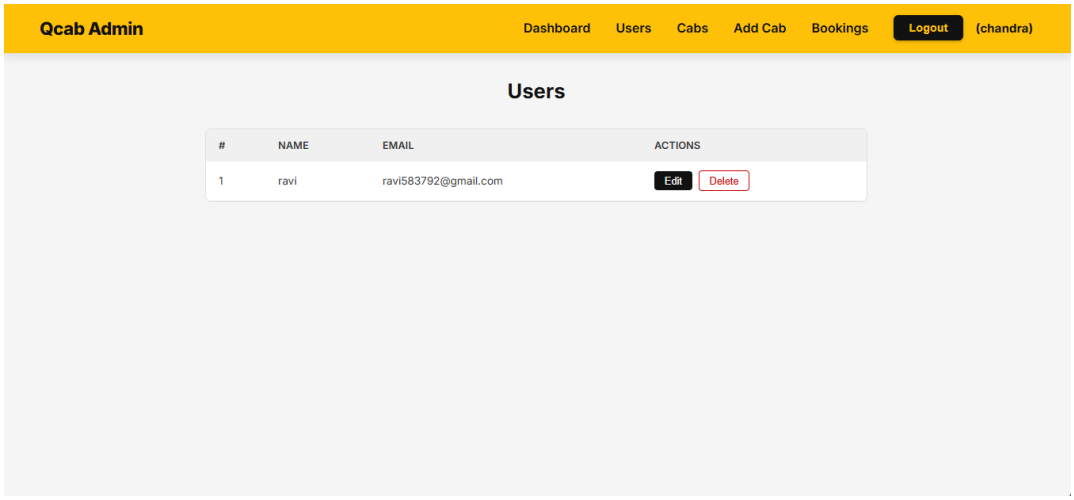
2. Admin Dashboard — Totals (Users, Cabs, Bookings)



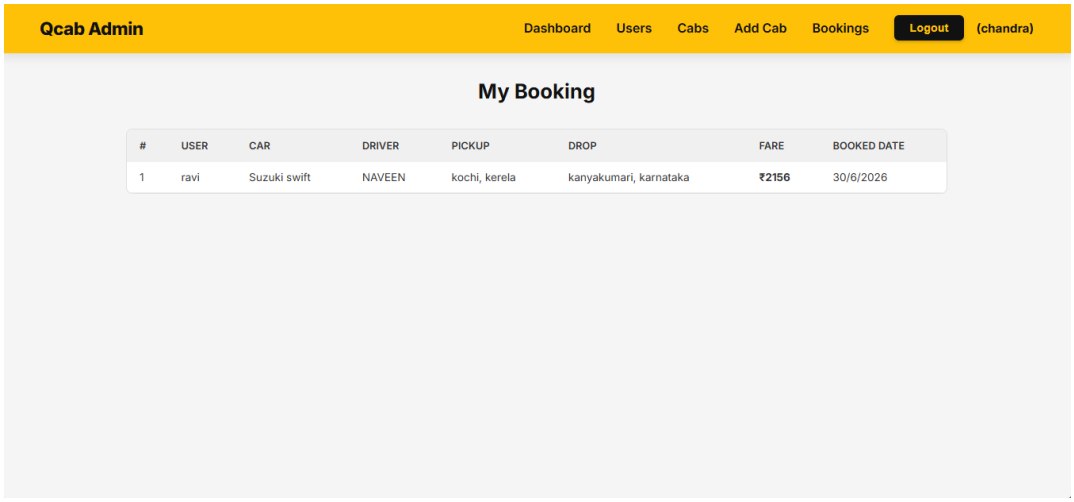
3. Admin Dashboard — Recent Cabs & System Summary



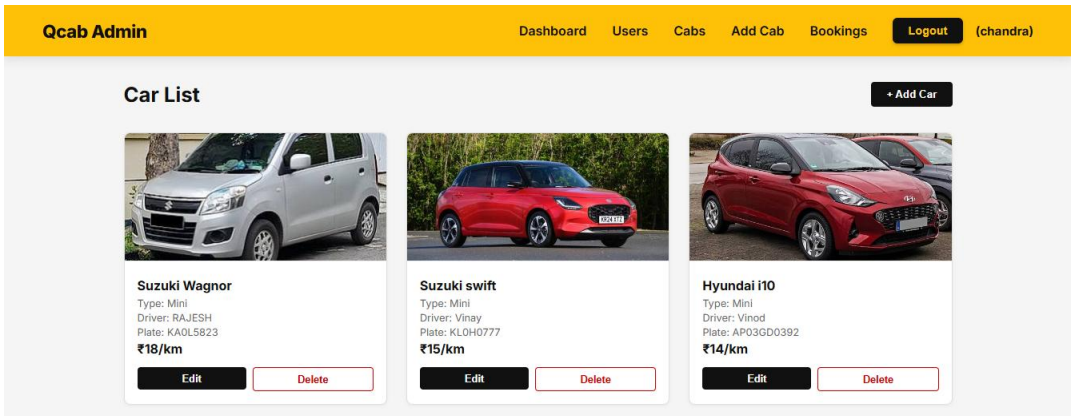
4. Admin — Users Management



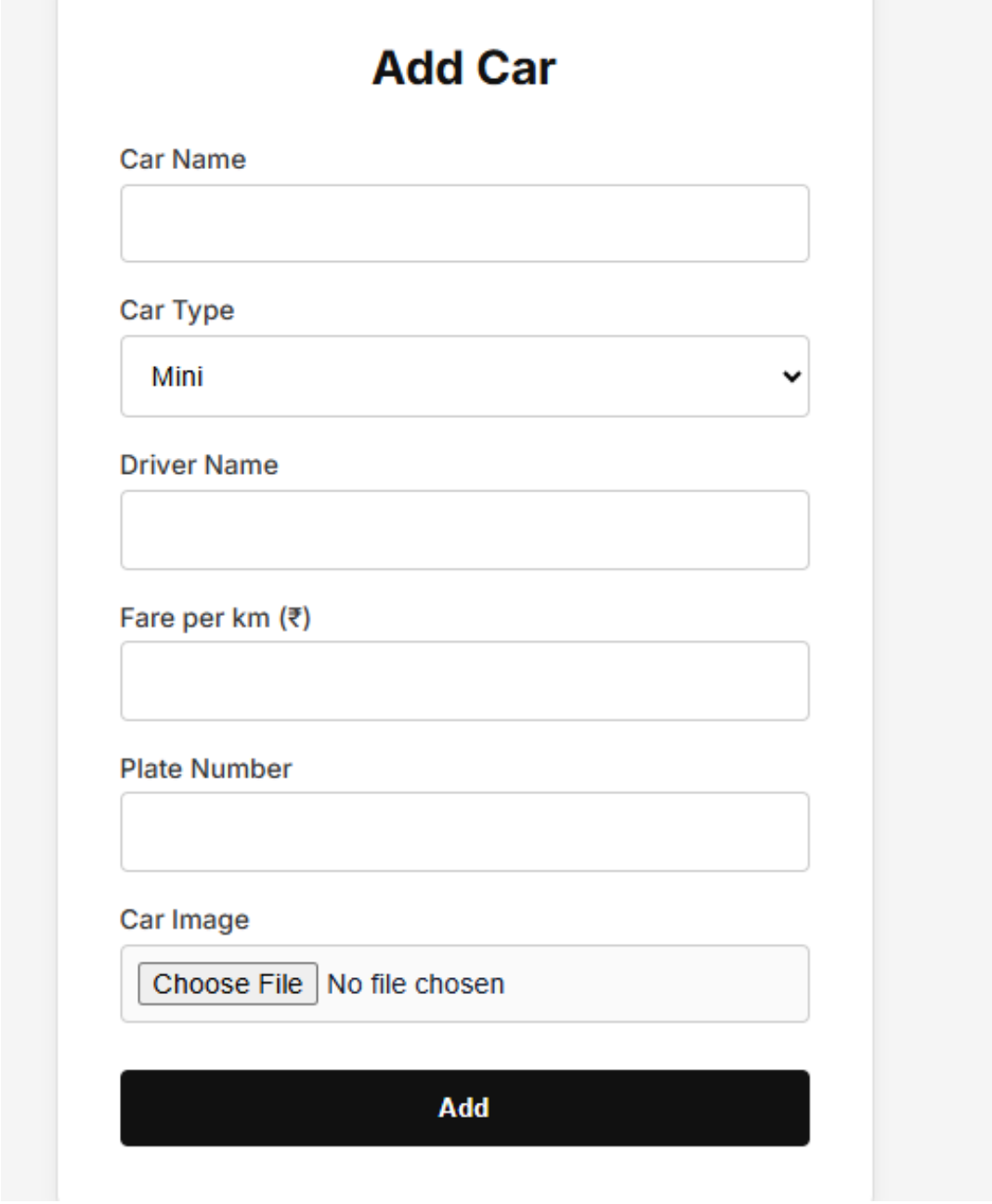
5. Admin — Bookings (All Users)



6. Admin — Car List (Manage Cabs)



7. Admin — Add Car (New Cab Form)



12. Known Issues

- The booking form does not yet validate that a drop location is selected before submission. There is no real-time driver tracking, in-app payment, ride cancellation, or driver-rating feature in the current build.

13. Future Enhancements

- Planned enhancements include real-time driver tracking on a map, in-app payment (UPI/card/wallet), ride cancellation, driver ratings and feedback, OTP-based login, and a dedicated mobile app.